# AI and Coding Workshop

Instructor: Karin Huangsuwan
3000747 Healthcare Innovation and Entrepreneurship

# AI is Getting Better Every Day

1997

## Optimization

Focused on solving closed-system puzzles. AI could calculate millions of permutations to find the "best" move in games like Chess.

2012

## Identifying Images

Neural networks allowed AI to "see." Machines began to categorize the world, identifying objects and faces with superhuman accuracy.

2017

## Understanding Language

AI moved beyond keywords to grasp semantics. It began to understand the nuances of human intent, sentiment, and complex instructions.

2022

## Writing and Generating

Generative models turned data into expression. AI became capable of drafting professional copy and rendering photorealistic imagery from scratch.

2023

## Reasoning

Models began to "think" through problems. This involved chain-of-thought processing to solve math, debug code, and justify its own decisions.

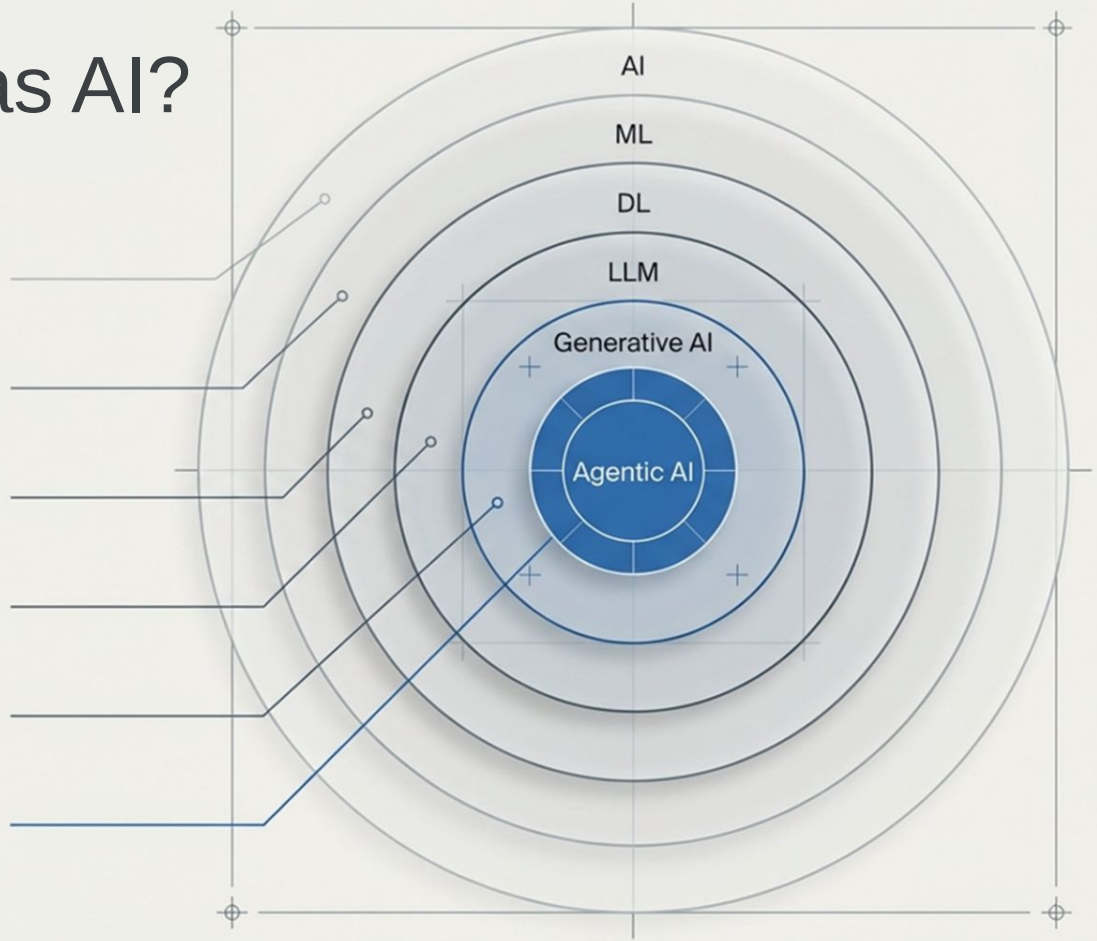
Today

## Autonomous Action

AI now executes multi-step workflows. It doesn't just suggest a plan; it uses tools, browses the web, and completes the task independently.

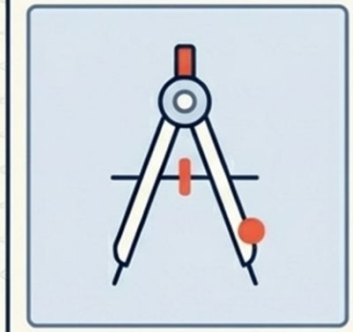
# What is classified as AI?

**Artificial Intelligence:** The broad field of making machines smart (e.g., a chess engine).
**Machine Learning:** Learning patterns from data instead of explicit programming.
**Deep Learning:** Neural networks powering modern image recognition and translation.
**Large Language Model:** Massive text training to predict the next word in a sequence.
**Generative AI:** Creating new content rather than classifying existing data.
**Agentic AI:** Using tools and making decisions to achieve a goal. Acting, not just talking.




NotebookLM

# Before Vibe Coding: Who Designs an AI App?



## Software Architect

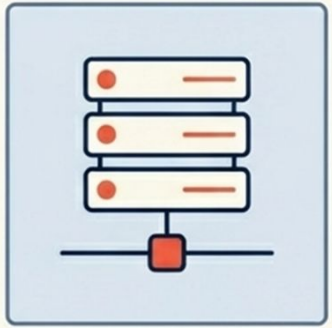
*The Urban Planner*

**Domain:** Tech stack, databases, and cross-service roadmaps.

## Software Engineer

*The Builder*

**Domain:** Algorithms, data structures, and feature implementation.



## DevOps / SRE

*The Grid Operator*

**Domain:** Server infrastructure, uptime, and traffic management.

## Product Manager

*The Visionary*

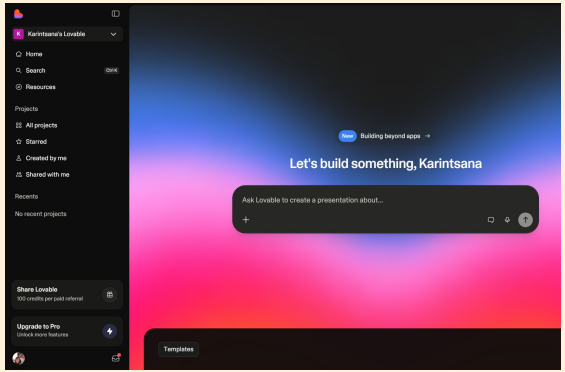
**Domain:** User needs and ensuring the architecture serves a human purpose.

NotebookLM

# After Vibe Coding: Code in English (or Any Language)

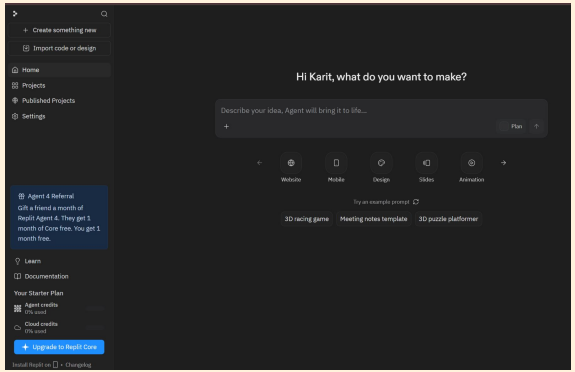
**Vibe coding** is a software development approach where

- you build applications primarily through natural language conversation with an AI
- rather than writing code line-by-line yourself.



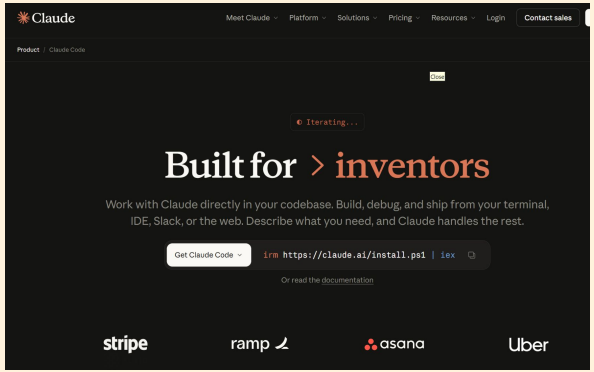

## Lovable: The Visual Prototyper

Best for non-coders and rapid MVPs. It "vibes" high-fidelity UI and auto-provisions **Supabase** backends from simple text or screenshots.




## Replit: The All-in-One Studio

The cloud-native powerhouse for **full-stack** apps. Its agent handles the code, database, and hosting in a single browser tab.




## Claude Code: The Terminal Agent

The developer's expert for existing projects. It operates directly on your **local files** to refactor, debug, and run tests via command line.

# How to Vibe Code?

1. **Prompting:** You describe what you want in plain English (e.g., *"Make a dashboard that tracks my stock portfolio and turns the background red if I'm losing money"*).
2. **Generation:** An AI tool (like **Cursor**, **Replit Agent**, or **Claude**) writes the entire block of code for you.
3. **Iterating:** You run the app, see what’s wrong, and "talk" the AI through the fix (e.g., *"The red is too bright, make it a soft pastel and add a 'Refresh' button"*).
4. **Testing:** Automated bug-finding and validation to ensure the app actually works.
5. **Deployment:** deployment to a live environment, moving from "thought" to "product."

# Current Advanced AI Tools

1. Large Language Models (LLM)
2. Computer Vision Models
3. Machine Learning Models

**Large Language Model:** A system that has "read" almost everything on the internet.

ChatGPT

Gemini

Claude

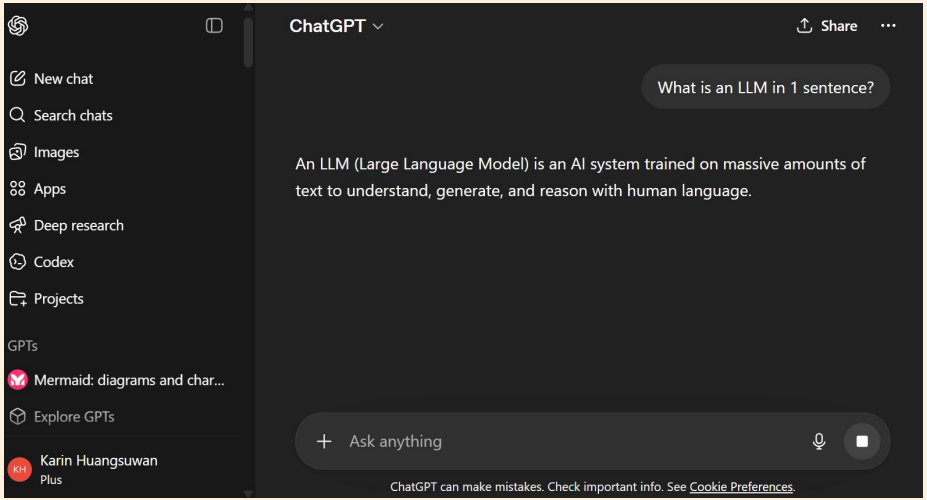
# Accessing Large Language Models

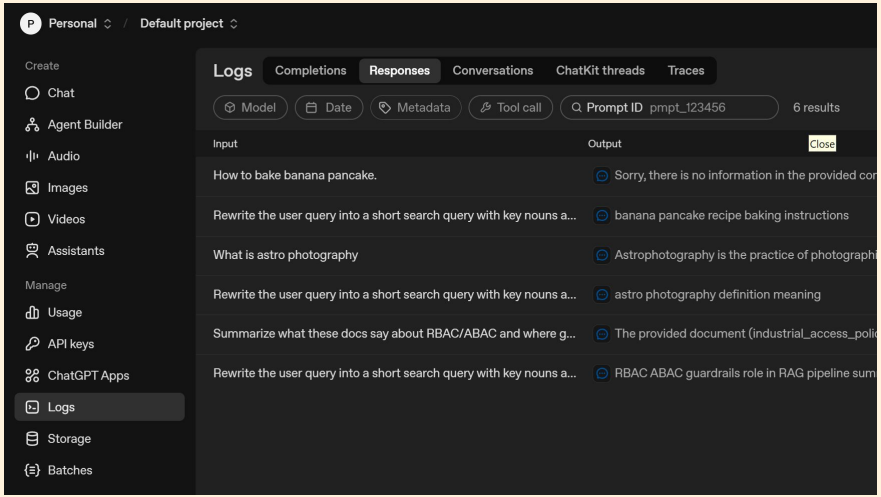
1. Web UI, good for: everyday use

https://chatgpt.com/
https://gemini.google.com/app
https://claude.ai

2. Developer API, good for: app development that involves LLM calling

http://platform.openai.com/
https://aistudio.google.com/
https://platform.claude.com/



# RAG System: Add Knowledge to LLM

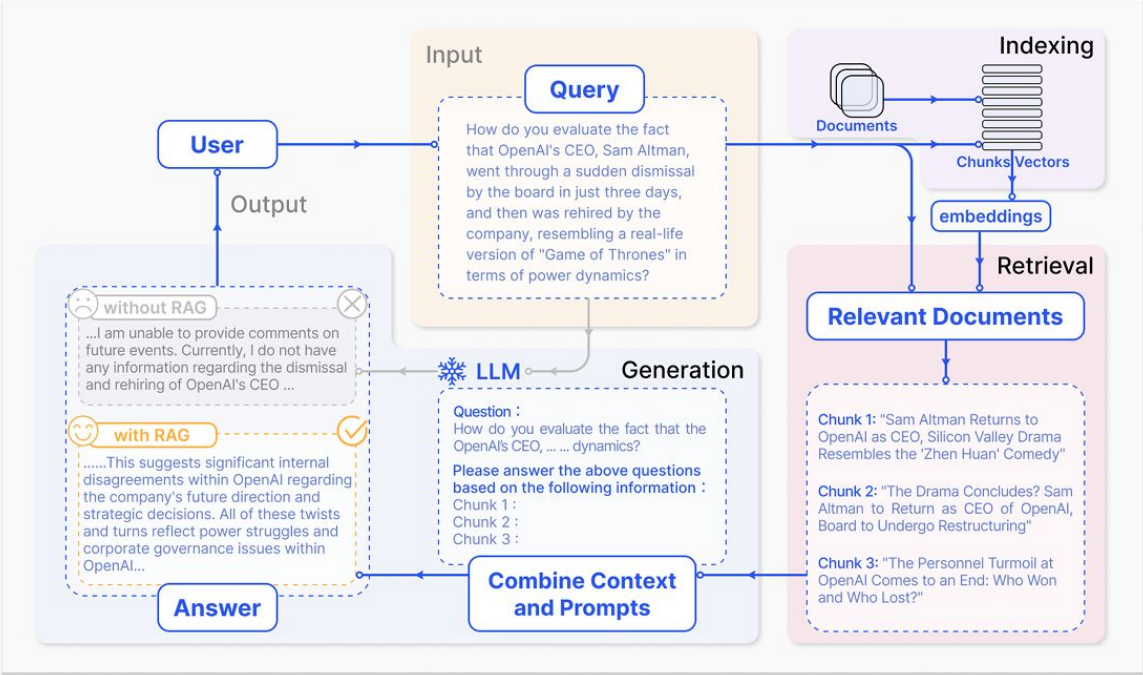
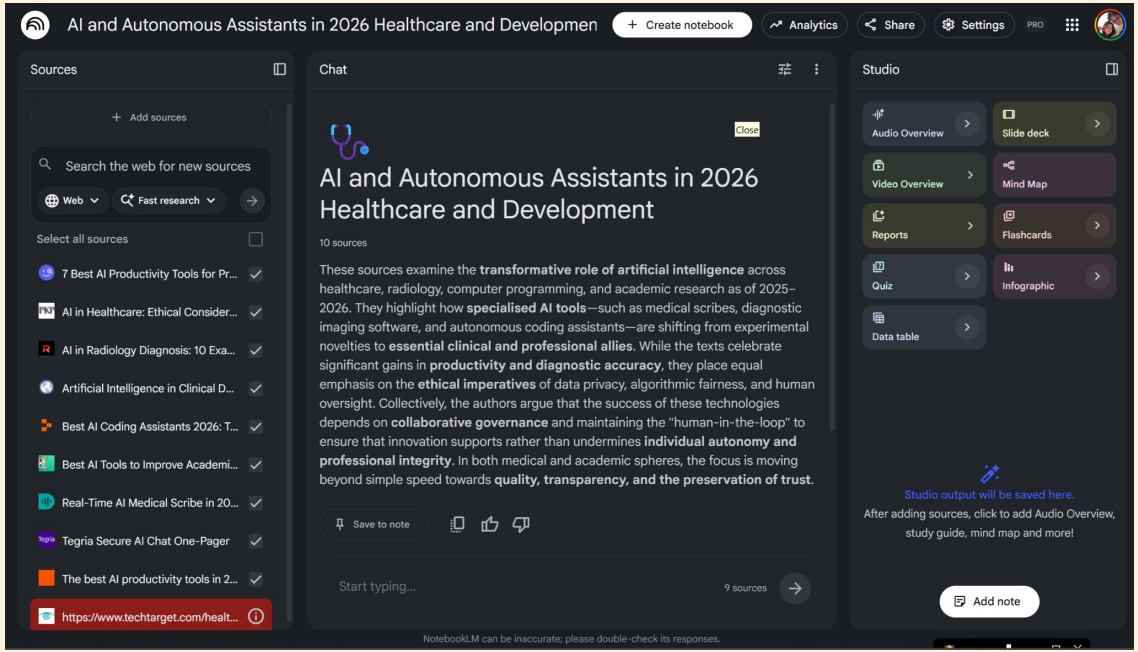



Fig. 2. A representative instance of the RAG process applied to question answering. It mainly consists of 3 steps. 1) Indexing. Documents are split into chunks, encoded into vectors, and stored in a vector database. 2) Retrieval. Retrieve the Top k chunks most relevant to the question based on semantic similarity. 3) Generation. Input the original question and the retrieved chunks together into LLM to generate the final answer.

# NotebookLM: Research Helper

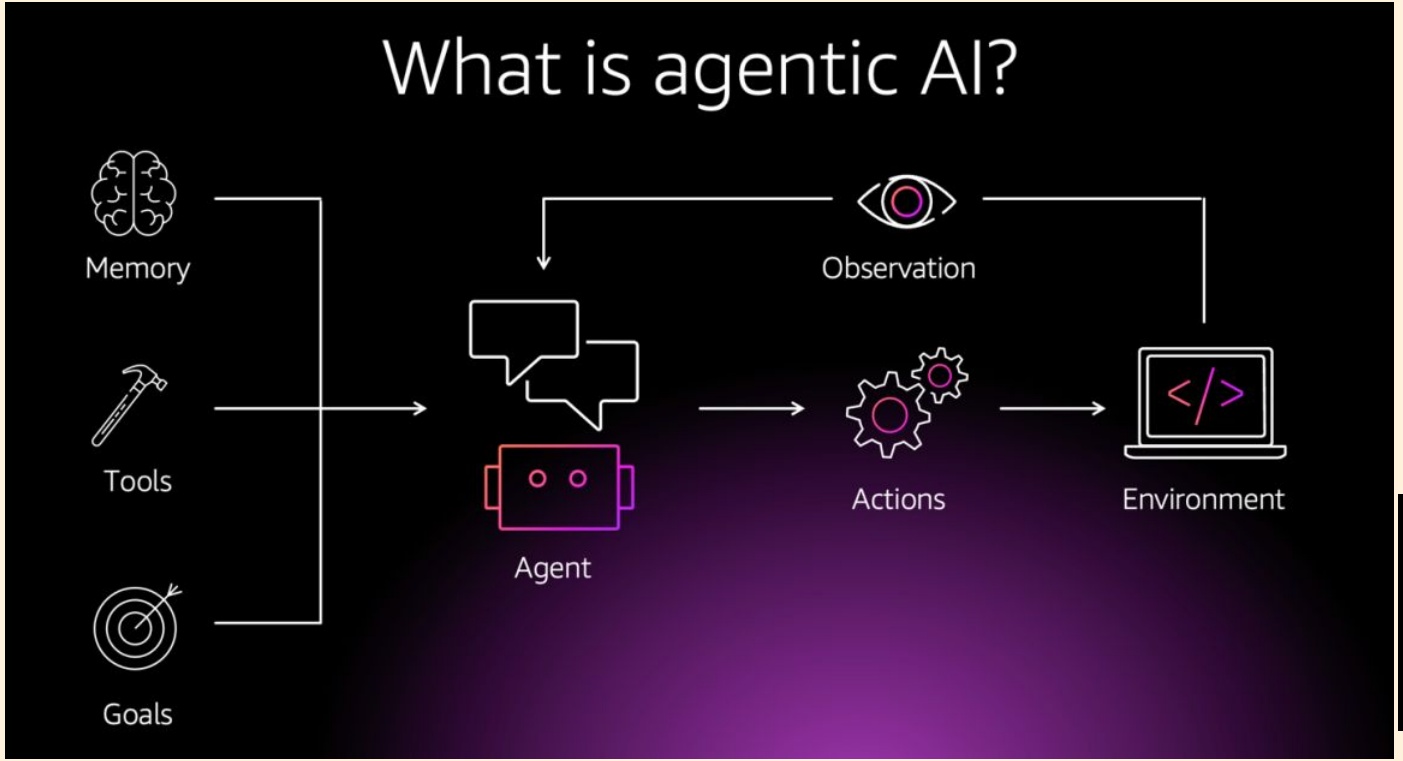


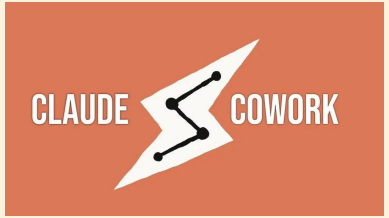

**Core Capabilities:**

- **Sources Panel:** Upload and manage your data. It supports a massive **1 million token context window** per notebook, allowing you to analyze dozens of documents, entire books, or hours of video simultaneously.
- **Chat Panel:** Ask questions, summarize complex topics, or brainstorm ideas specifically based on your sources. It provides inline citations so you can verify exactly where the information came from.
- **Studio Panel:** Instantly transform your research into "artifacts" like reports, slide decks, and data tables.

**https://notebooklm.google.com/**

# Agentic AI: The Future of Workflow Automation

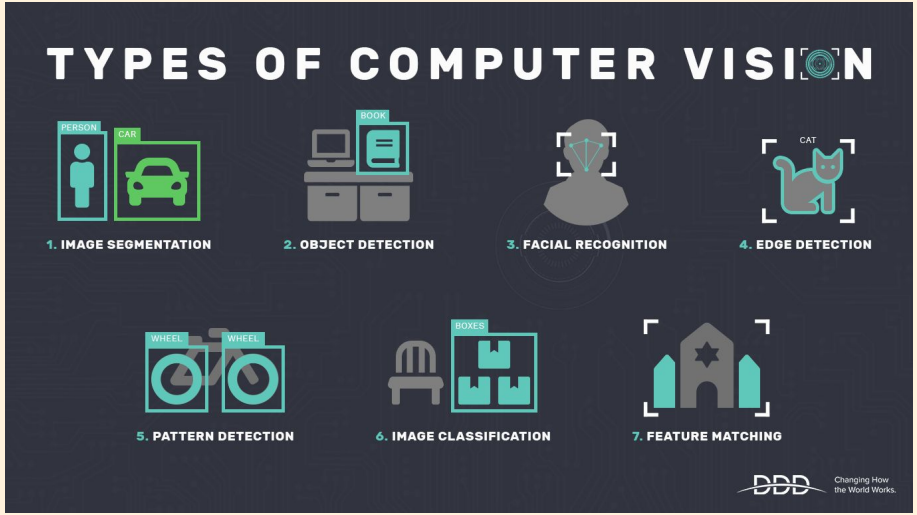


https://aws.amazon.com/blogs/machine-learning/build-agentic-systems-with-crewai-and-amazon-bedrock/

# Deep Learning Models

**Computer Vision Models:** AI systems designed to interpret and understand visual data from the world, such as images and videos, by identifying patterns and features.




https://www.digitaldividedata.com/blog/everything-about-computer-vision

Open-source hubs for computer vision models:

- **Hugging Face (Computer Vision Hub):** huggingface.co/models?pipeline_tag=image-classification
  - *Best for:* The widest variety of modern models (Transformers, Diffusion, etc.) and easy integration with Python.
- **PyTorch Hub:** pytorch.org/hub
  - *Best for:* Research-ready models that can be loaded with a single line of code (`torch.hub.load`).
- **TensorFlow Hub (now Kaggle Models):** kaggle.com/models/google/resnet-v2
  - *Best for:* Models optimized for production and mobile deployment (TensorFlow Lite).
- **Papers with Code:** paperswithcode.com/area/computer-vision
  - *Best for:* Finding the "State of the Art" (SOTA) for specific tasks like object detection or segmentation, linked directly to their GitHub code.
- **Model Zoo (OpenCV):** github.com/opencv/opencv/wiki/Model-Zoo
  - *Best for:* Finding lightweight models specifically optimized to run fast on CPUs or edge devices.

# Integrating Open-Source CV Models into Your App

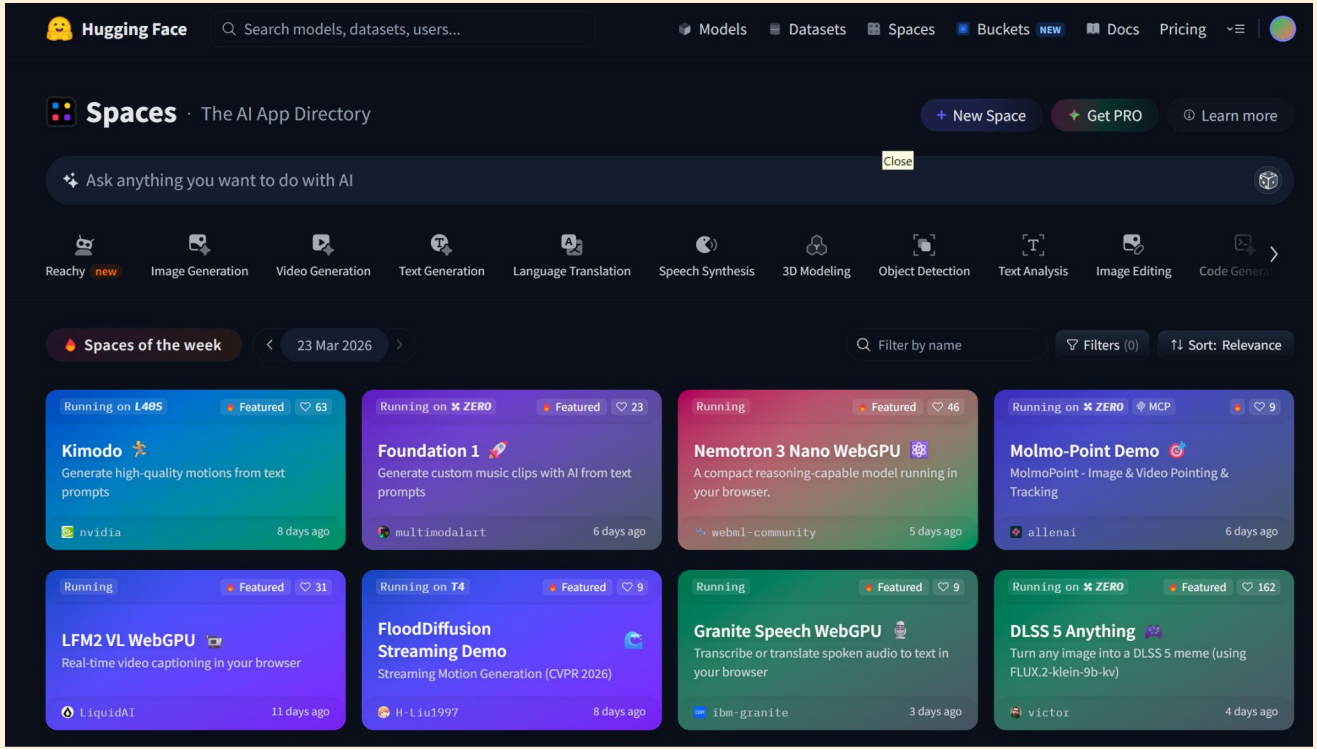
1. Choose your hosting environment (where the model will run on)
   a. Cloud Hosting (Server-Side): The model runs on a powerful remote server (GPU-enabled). Your app sends an image via an API and gets a result back.
   b. Edge/On-Device Hosting: The model is converted and compressed to run directly on the user’s phone or computer.
2. Prepare the Model for Hosting

- Pickling/Saving: Saving the weights (e.g., .pth for PyTorch, .h5 for TensorFlow).

3. Setting up the API

- Containerization: Use Docker to package the model, its libraries (OpenCV, PyTorch), and the code into one portable "container."
- Web Frameworks: Wrap the model in a FastAPI or Flask script. This creates a URL (e.g., https://api.myapp.com/predict) that your app can "POST" images to.

# Free Hosting Environment: Huggingface Spaces




A **Hugging Face Space** is a free, hosted environment that allows you to turn your AI model into a live, interactive web application or API in minutes.

https://huggingface.co/spaces